IILM UNIVERSITY
Gurugram | Greater Noida

INTERNSHIP PRESENTATION

Python Developer Internship

LAKSHYA TRIPATHI
Roll No. 25SCS1003001040 • Section 2CSE32
Semester 3RD

Submitted to IILM University, Greater Noida, Uttar Pradesh

04 **WEEKS**

06 AUGUST — 06 AUGUST 2026

CODEC
Technologies • Python Internship

INTERNSHIP AT A GLANCE

Four weeks strengthened practical Python skills

The internship progressed from Python fundamentals and advanced concepts to data handling, APIs and industry-oriented project work.

100%
Portal completion

04
Learning weeks

02
Project briefs

Host: Codec Technologies • Role: Python Intern • August 2026

STUDENT PROFILE

Lakshya Tripathi

Roll number
25SCS1003001040

Section
2CSE32

E-mail id

Semester 3RD

LEARNING ROADMAP

Five milestones built an end-to-end Python workflow

The curriculum moved from core syntax and reusable functions to OOP, data processing, APIs and project implementation.

WEEK 1	WEEK 2	WEEK 3A	WEEK 3B	WEEK 4
01	02	03	04	05
Python basics	**Advanced Python**	**Data handling**	**APIs & web**	**Project delivery**
Syntax, loops, functions	OOP, files, exceptions	CSV, JSON, Pandas	Requests, Flask/FastAPI	Build, test, document

LEARNING LOGIC **Fundamentals → modular code → data → APIs → project delivery**

Python fundamentals established a reliable coding base

fundamentals.py

```
name = input('Name: ')
marks = [78, 84, 91]
average = sum(marks) / len(marks)
for score in marks:
  print(score)
```

Core practices

- Use variables and built-in data types correctly
- Apply conditions and loops to control program flow
- Write reusable functions with clear inputs and outputs
- Organize code for readability and simple debugging

EXPECTED RESULT

Small command-line programs that solve structured problems with predictable results.

Advanced Python made programs easier to maintain

Object-oriented design, file operations and exception handling improved structure and reliability.

OOP — Reusable classes and objects

FILES — Read and write persistent data

ERRORS — Safer execution with try and except

MODULES — Organized code through imports

student_record.py

```
class Student:
  def __init__(self, name):
    self.name = name
try:
  record = Student('Lakshya')
except ValueError as error: ...
```

OUTCOME • Programs became easier to extend, reuse and troubleshoot

Data handling turned records into insight

data_analysis.py

```
import pandas as pd
df = pd.read_csv('records.csv')
df = df.dropna()
df['score'] = df['score'].astype(int)
summary = df.groupby('course').mean()
```

Data workflow

- Load structured data from CSV and JSON
- Clean missing or inconsistent values
- Transform rows with reusable functions
- Summarize results for reporting or export

KEY LEARNING • Validate data before drawing conclusions or saving output

API practice connected Python to live services

The learning module introduced web requests, JSON responses and lightweight endpoints for application integration.

01 **SELECT**
Endpoint and parameters chosen

02 **REQUEST**
HTTP request sent

03 **CHECK**
Status and errors validated

04 **PROCESS**
JSON data transformed

api_client.py

```
response = requests.get(url, timeout=10)
response.raise_for_status()
payload = response.json()
return payload
```

OUTPUT • Structured API data made available to a Python application

Two project briefs combined Python skills

The final stage applied Python, NLP, document processing and lightweight web frameworks to practical application concepts.

01

DEFINE

Clarify user need

02

PROCESS

Clean text inputs

03

MODEL

Build NLP pipeline

04

SERVE

Flask or FastAPI

05

DOCUMENT

Results and usage

PROJECT FOCUS • AI-powered chatbot | Automated resume parser

TECHNICAL TOOLKIT

The toolkit covered the Python workflow

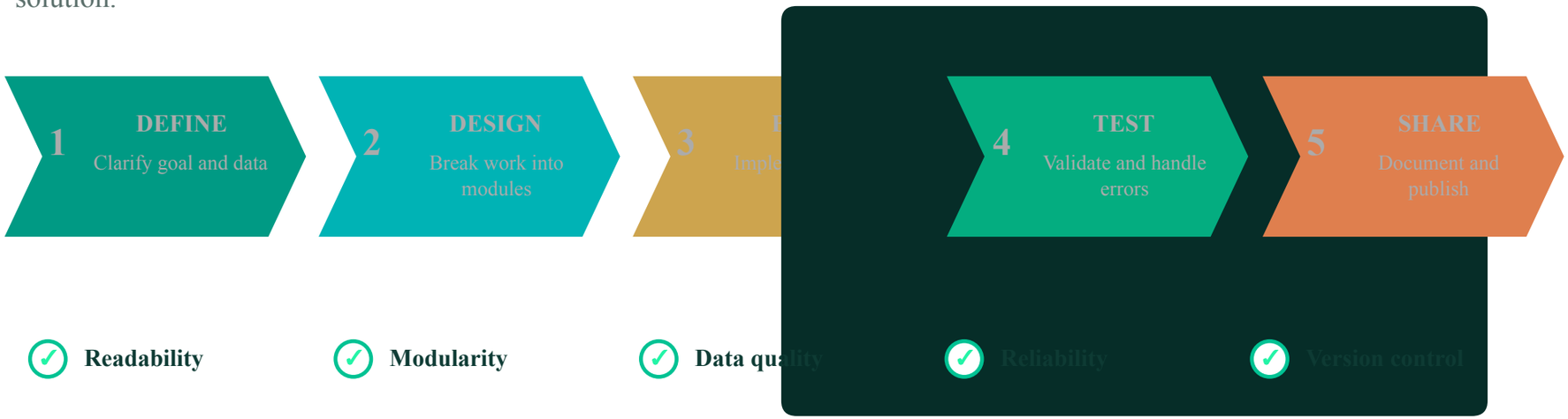
FOUNDATION	**Core Python**	Syntax • data types • control flow • functions
DESIGN	**Advanced**	OOP • files • modules • exception handling
DATA	**Analysis**	Pandas • CSV • JSON • SQL and SQLite
DELIVERY	**Web and NLP**	Requests • Flask/FastAPI • NLTK/spaCy • GitHub

Tools were practiced as connected layers, from clean code to usable applications.

SOLUTION WORKFLOW

A repeatable workflow linked problem to delivery

This workflow reflects the progression from understanding a requirement to validating and sharing a Python solution.



PRINCIPLE • Build, test, then integrate the solution

Practice targeted common Python project risks

The modules highlighted practical issues that arise when scripts move from examples to reusable applications.

CHALLENGE	TECHNICAL RESPONSE	INTENDED EFFECT
INPUTS VARY	Validate types, formats and missing values	**Fewer avoidable processing failures**
LOGIC GROWS	Split work into functions, classes and modules	**Easier maintenance and testing**
APIS FAIL	Add timeouts, status checks and exceptions	**More reliable service integration**
OUTPUTS DRIFT	Use test cases and clear result formats	**Consistent, explainable behavior**

These responses summarize the curriculum and project requirements; no unsupported performance metrics are claimed.

PROJECT OUTCOMES

Four weeks show growing Python capability

The portal records 100% course completion and concludes with two industry-oriented project requirements.

01 **Foundation scripts**
Syntax, conditions, loops, functions

02 **Modular programs**
OOP, files, exceptions, modules

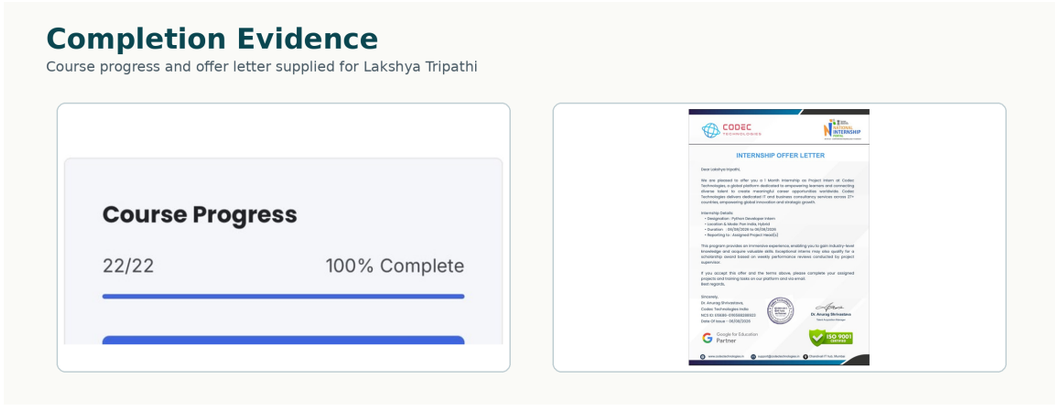
03 **Data workflows**
Pandas, CSV, JSON, SQL

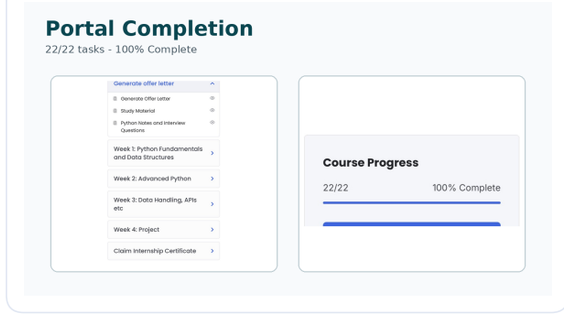
04 **API integration**
Requests, JSON, Flask/FastAPI

05 **Project delivery**
Chatbot and resume-parser concepts

100% COMPLETE

Attached records confirm internship evidence





100%

INTERNSHIP PROGRESS

The portal and offer letter confirm Lakshya Tripathi's Python Developer Internship at Codec Technologies.

SKILLS GAINED

The internship built technical delivery skills

TECHNICAL DEPTH

Build Python solutions with clarity and confidence

- Write readable, modular Python
- Work with files and structured data
- Consume APIs and validate responses
- Apply NLP and document-processing concepts
- Present and document project outcomes

PROFESSIONAL HABITS

01 Progressive problem solving

Break requirements into clear, testable steps.

02 Self-paced learning

Complete structured modules independently.

03 Quality awareness

Use validation, exceptions and clear outputs.

04 Professional communication

Document work and prepare it for professional sharing.

IILM UNIVERSITY
Gurugram | Greater Noida

ete
rkflow

ar programming,
ical learning

LAKSHYA TRIPATHI • 25SCS1003001040 • 2CSE32
Semester 3RD

NEXT APPLICATION

01 **Build larger Python projects**

02 **Strengthen testing skills**

03 **Publish documented work**

Acknowledgement

With gratitude to IILM University and Codec Technologies for enabling this practical learning experience.